CiT Backend Engineering Track

Week 16: Messaging & Event-Driven Architecture

Why Asynchronous Processing, Kafka Core Concepts & Your First Producer and Consumer

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 16
Instructor	Jim & Anderson
Duration	6 months | 24 weeks
Tech Stack	Java 25, Spring Boot 4, Spring Kafka, Apache Kafka, Docker

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we begin a new chapter of our learning. We thank You for bringing us this far, and we ask for fresh energy for what is new. Teach us that not every good thing arrives at once — that some work is sent out now and fulfilled later, in its proper time. Give us patience to wait well, and faithfulness to do our part while we wait.

As Habakkuk 2:3 reminds us, "Though it tarries, wait for it; because it will surely come, it will not tarry." Lord, help us trust Your timing in our studies, our work, and our lives.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,
Amen.

Scripture for the Week

"For the vision is yet for an appointed time; but at the end it will speak, and it will not lie. Though it tarries, wait for it; because it will surely come, it will not tarry."

— Habakkuk 2:3 (NKJV)

Table of Contents

1. Welcome to Week 16
2. Why Asynchronous Processing?
3. Events & Messaging — The Core Ideas
4. Apache Kafka — Topics, Partitions & Offsets
5. Producers, Consumers & Consumer Groups
6. Running Kafka Locally
7. Your First Spring Kafka Producer & Consumer
8. Designing Good Events
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 16

Over the last seven weeks you have built a professional, secure REST API: cleanly layered, well-modelled, validated, and protected by Keycloak. Every piece of it has worked the same way — a request arrives, the API does all the work, and then it replies. This week you learn that not everything should happen that way.

This begins the Messaging & Event-Driven Architecture module. When a member borrows a book, the loan must be saved immediately. But sending them an SMS, emailing a receipt, and updating the library's reports do not need to hold up their request. Those jobs can be announced as an event and handled in the background by other parts of the system.

The tool that carries those events is Apache Kafka — the event-streaming backbone of the XENO platform. This week you will learn why systems work this way, how Kafka is organised, and you will run Kafka on your own machine and send your first events from Spring Boot.

What to Expect in Week 16

By the end of this session, you will be able to:

- Explain why some work should happen asynchronously, and what it costs
- Describe events, producers, consumers, and brokers in plain English
- Explain Kafka topics, partitions, and offsets
- Explain how message keys, ordering, and consumer groups work together
- Run Kafka locally and send and read messages from the command line
- Publish events with KafkaTemplate and consume them with @KafkaListener
- Design clear, safe events that follow the platform conventions

2. Why Asynchronous Processing?

Before learning Kafka, understand the problem it solves. It begins with a request that tries to do too much.

2.1 The Request That Does Everything

Here is a borrow-a-book use case written the way you might write it today. The facade (Week 8) does the real job, then three extra jobs, one after another:

```
public LoanResponse borrow(String memberId, Long bookId) {
    Loan loan = loanService.create(memberId, bookId);   // the real job
    smsGateway.sendReminder(loan);                      // slow, sometimes down
    emailService.sendReceipt(loan);                     // slow
    reportService.recordLoan(loan);                     // another system
    return loanMapper.toResponse(loan);
}
```

The member waits for all four steps. If the SMS gateway takes eight seconds, borrowing takes eight seconds. If it is down, the whole request fails — even though the loan itself was perfectly fine. And every new requirement ("also update the recommendations") makes this method longer and more fragile.

2.2 Do the Essential Work Now; Announce the Rest

The better design saves the loan — the one thing that must happen now — and then simply announces "a loan was created". Separate parts of the system hear that announcement and do their own jobs in their own time.

> **The Order Slip in the Kitchen**
>
> In a busy restaurant, the waiter does not stand in the kitchen waiting for each meal. They pin an order slip on the rail and go back to serve other tables. The cooks pick up slips as they are ready. If one cook is slow, the waiter is not stuck. Kafka is the rail: producers pin up events, and consumers pick them up when they can.

2.3 What You Gain — and What It Costs

Benefit	What It Means

Faster responses	The member gets an answer as soon as the loan is saved
Resilience	If the SMS service is down, loans still work; the SMS is sent when it recovers
Load levelling	A burst of activity — every parent paying fees at the start of term — is absorbed and processed steadily
Easy to extend	A new service can start listening for loan events without changing the loan code at all

Asynchronous Is Not Free

There is a cost. The SMS now arrives a few seconds after the loan, not at the same instant — this is called eventual consistency. Problems are harder to trace because the work happens in several places. And a message can occasionally be delivered twice. Next week you will learn to handle all of this. Use messaging for work that can safely happen a little later — never for the essential step the user is waiting on.

Exercise 1: Now or Later?

For each step of a school fees payment, decide whether it must happen inside the request (now) or can be an event (later), and give one reason:

1. Record the payment of UGX 450,000 against the student's account
2. Send the parent an SMS receipt
3. Check the parent has enough balance before paying
4. Update the bursar's daily collections report
5. Email the class teacher that fees are cleared

3. Events & Messaging — The Core Ideas

Messaging has a small vocabulary. Learn these words precisely and everything else in the module will make sense.

3.1 What Is an Event?

An event is a record of something that has already happened. Because it describes the past, it is named in the past tense: LoanCreated, BookReturned, PaymentReceived. An event is a fact — it cannot be refused, only reacted to.

Contrast that with a command, which asks for something to happen: "SendSms", "CreateLoan". A command can be rejected. In event-driven systems, services mostly announce facts and let others decide how to respond.

3.2 The Three Players

Player	Its Job
Producer	Publishes events. Here, the Library API publishes LoanCreated.
Broker	Stores events safely and delivers them. Here, Kafka.
Consumer	Reads events and reacts. Here, a notification service that sends the SMS.

3.3 Two Styles of Messaging

Message queue	Event stream (log)
Each message goes to one worker, then is removed	Each event is kept, and every interested service reads it
Good for sharing out jobs ("process this upload")	Good for announcing facts many services care about
Example: RabbitMQ	Example: Apache Kafka

The XENO platform uses both: Kafka is the primary event-streaming backbone, and RabbitMQ handles classic queue patterns. This module focuses on Kafka.

The Producer Does Not Know Who Is Listening

This is the heart of the idea. The Library API publishes LoanCreated and moves on. It does not know — or care — whether one service is listening or ten. Next month a reporting team can start consuming loan events without anyone touching the loan code. That independence is called decoupling.

4. Apache Kafka — Topics, Partitions & Offsets

Apache Kafka is a distributed event-streaming platform: it stores streams of events safely and delivers them to any number of consumers. Three ideas explain how it is organised.

4.1 Topics — A Named Log of Events

Events are stored in a topic — a named stream such as library.loan-created. A topic is an append-only log: new events are always added at the end, and existing events are never edited.

Crucially, reading an event does not remove it. Kafka keeps events for a retention period you configure — seven days, say — so a new consumer can start from the beginning and replay history. This is the big difference from a queue.

> **The Ledger Book**
> Think of a ledger book at a school bursar's office. Each payment is written on the next numbered line, in order, and is never rubbed out. The bursar, the head teacher, and an auditor can each read the book at their own pace, from any line they choose. A Kafka topic is that ledger.

4.2 Partitions — Splitting the Work

One ledger can only be read so fast. So a topic is split into partitions — several separate logs that together make up the topic. Different consumers can read different partitions at the same time, which is how Kafka handles enormous volumes.

```
Topic: library.loan-created   (3 partitions)

Partition 0:  [0] [1] [2] [3] [4]  →  new events appended here
Partition 1:  [0] [1] [2]          →
Partition 2:  [0] [1] [2] [3]      →
```

4.3 Offsets — Knowing Where You Are

Each event in a partition has a position number called its offset — the line number in the ledger. A consumer remembers the offset it has reached, like a bookmark, so if it stops and restarts it carries on exactly where it left off.

Term	Plain English

Broker	One Kafka server that stores partitions
Cluster	Several brokers working together
Replication	Copies of each partition kept on other brokers, so no data is lost if one fails
Retention	How long Kafka keeps events before deleting old ones

5. Producers, Consumers & Consumer Groups

Now see how events get into Kafka, how they come out, and how Kafka lets you scale consumers safely.

5.1 Keys Decide the Partition

A producer sends each event to a topic, optionally with a key. Kafka uses the key to choose the partition, and all events with the same key always go to the same partition. Because each partition is strictly ordered, events with the same key are always read in the order they were sent.

So if you use the member id as the key, all of one member's loan events stay in order: borrowed, then renewed, then returned. Events for different members can be processed in parallel.

Ordering Is Only Within a Partition

Kafka guarantees order inside one partition, not across a whole topic. If two events must be handled in order, give them the same key. Choose keys deliberately — usually the id of the thing the events are about.

5.2 Consumer Groups — Sharing and Scaling

Consumers join a consumer group by sharing a group id. Kafka divides the topic's partitions among the members of the group, so each partition is read by exactly one consumer in that group. Add more consumers, and the work is shared out further.

Different groups are completely independent: each group receives every event. That is how one LoanCreated event can reach both a notification service and a reporting service.

Consumer group	Instances	What happens
notification-service	3	Each instance reads one of the 3 partitions — the load is shared
reporting-service	1	The single instance reads all 3 partitions and also sees every event

More Consumers Than Partitions?

A partition can only be read by one consumer in a group at a time. With 3 partitions, a fourth instance in the same group would sit idle, waiting to take over if

another fails. The number of partitions sets the limit on how far one group can scale — choose it with growth in mind.

Exercise 2: Reason About a Topic

A topic fees.payment-received has 4 partitions and is keyed by student id. Answer:

6. Will two payments for the same student be processed in order? Why?
7. A receipts service runs 2 instances in one group. How many partitions does each read?
8. An analytics service starts tomorrow with its own group. Can it read last week's payments?
9. What happens if the receipts service runs 6 instances?

6. Running Kafka Locally

Concepts become real once you can see them. Let us run a Kafka broker on your own machine and send events through it by hand.

6.1 Start a Broker

As with Keycloak last week, the quickest way is Docker (studied properly in Week 18). Current Kafka runs in KRaft mode and needs no separate ZooKeeper server — older tutorials that start ZooKeeper first are out of date.

```
docker run -d --name kafka -p 9092:9092 apache/kafka:latest
```

Your broker is now listening on localhost:9092.

6.2 Create a Topic

Kafka ships with command-line tools inside the container. Create a topic with three partitions:

```
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic library.loan-created \
  --partitions 3 --replication-factor 1
```

Replication factor 1 means one copy, which is fine for a single broker on your laptop. Production clusters keep three copies.

6.3 Send and Read Events by Hand

Open two terminals. In the first, start a console consumer:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic library.loan-created --from-beginning
```

In the second, start a console producer and type a few lines, pressing Enter after each:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 --topic library.loan-created
```

Each line you type appears in the consumer a moment later. You have just produced and consumed your first events.

Exercise 3: See Replay for Yourself

10. Send five messages with the console producer
11. Stop the consumer, then start a brand-new one with --from-beginning
12. Notice that all five messages appear again — reading never removed them
13. Explain in one sentence why a queue would have behaved differently

7. Your First Spring Kafka Producer & Consumer

Now let the Library API publish a real event when a loan is created, and let a listener react to it. Spring for Apache Kafka makes both remarkably short.

7.1 Dependency and Configuration

Add the Spring for Apache Kafka dependency (spring-kafka — choose "Spring for Apache Kafka" on start.spring.io). Then point Spring at your broker. This week we send events as plain strings; next week you will send JSON objects.

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
```

auto-offset-reset: earliest means a brand-new group starts from the oldest event it can find, rather than only new ones.

7.2 Declare the Topic

In your configuration package, declare the topic as a bean so Spring creates it on start-up if it does not exist:

```
@Bean
NewTopic loanCreatedTopic() {
    return TopicBuilder.name("library.loan-created")
            .partitions(3)
            .replicas(1)
            .build();
}
```

7.3 The Producer

Producers and consumers live in the messaging package — the XENO package layout you adopted in Week 8. A producer is a bean that uses KafkaTemplate:

```
@Component
public class LoanEventProducer {

    private final KafkaTemplate<String, String> kafkaTemplate;

    public LoanEventProducer(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void loanCreated(String memberId, String bookTitle) {
        String message = "Member " + memberId + " borrowed " + bookTitle;
        kafkaTemplate.send("library.loan-created", memberId, message);
    }
}
```

The three arguments are the topic, the key (the member id, so each member's events stay in order), and the value. The facade calls it right after saving the loan — replacing the slow SMS, email, and report calls from Section 2.

7.4 The Consumer

```
@Component
public class LoanNotificationListener {

    private static final Logger log =
            LoggerFactory.getLogger(LoanNotificationListener.class);

    @KafkaListener(topics = "library.loan-created", groupId = "notification-service")
    public void onLoanCreated(String message) {
        log.info("Sending SMS: {}", message);   // the real SMS call comes later
    }
}
```

What Just Happened

A member borrows a book. The facade saves the loan and calls loanCreated — which hands the event to Kafka in a few milliseconds and returns. The member gets their answer at once. Moments later, the listener receives the event and "sends" the

SMS. If the SMS service were down, the event would simply wait in Kafka until it recovered.

Exercise 4: Publish Your First Event

Add the producer and listener above to your Library API. Borrow a book through Postman and find the "Sending SMS" line in your logs. Then watch the same event arrive in a console consumer from Section 6 — two independent readers of one event.

8. Designing Good Events

Sending an event is easy. Designing events that other teams can rely on for years takes care — because once other services consume your events, their shape becomes a contract, just like your OpenAPI spec.

8.1 Name Topics Clearly

In this course, name topics <domain>.<event-in-past-tense>, lowercase with hyphens: library.loan-created, library.book-returned, fees.payment-received. The name alone should tell a stranger what fact the topic records.

8.2 Give Every Event a Consistent Shape

Wrap each event in the same structure, using the same JSON conventions as your API (Week 10) — snake_case field names, UPPER_SNAKE_CASE values, and UTC timestamps:

```
{
  "event_id": "7d0c2e5a-1b3f-4d8e-9a61-2f4c8b7e1d90",
  "event_type": "LOAN_CREATED",
  "occurred_at": "2026-10-21T09:15:00Z",
  "data": {
    "loan_id": 1042,
    "member_id": "f3a1c9e2-7b4d-4c1a-9e55-2d8b6a0c1f77",
    "book_id": 77,
    "due_date": "2026-11-04"
  }
}
```

Field	Why It Is There
event_id	A unique id, so a consumer can spot a duplicate (vital next week)
event_type	What happened, so consumers can route and filter
occurred_at	When it happened, in UTC
data	The facts a consumer needs — ids and key values, nothing more

Events Are Read by Many — Keep Secrets Out

Events are kept for days and read by many services. Never put passwords, tokens, or personal data a consumer does not need into an event (Week 14). Prefer ids — a consumer that needs a phone number can look it up with proper permission.

8.3 Carry the Correlation ID

On the XENO platform, every Kafka message must carry an X-Correlation-Id header. It is the same id logged with the HTTP request that caused the event, so one search can follow an operation from the API call, through Kafka, to every consumer. Consumers that call other services or publish further events pass the same id on.

```
String correlationId = MDC.get("correlationId");   // set by the request filter

ProducerRecord<String, String> record =
      new ProducerRecord<>("library.loan-created", memberId, payload);
record.headers().add("X-Correlation-Id",
      correlationId.getBytes(StandardCharsets.UTF_8));

kafkaTemplate.send(record);
```

A Glimpse Ahead — Local Copies of Other Services' Data

Events also let one service keep a read-only copy of another service's data. On the XENO platform such tables are named with a _matview suffix (for example book_matview) to make clear they are maintained by a Kafka consumer, not owned by the service. You will see this pattern again as systems grow.

Exercise 5: A Proper LoanCreated Event

14. Change your producer to send the JSON shape above as the message value (as a string for now)
15. Use the member id as the key, and add the X-Correlation-Id header
16. In the listener, log the event_id and event_type you received
17. Document the topic, its key, and its payload in your project README

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Why some work should be asynchronous — faster responses, resilience, load levelling — and its costs
- Events are past-tense facts; producers publish them, brokers store them, consumers react
- Queues share out jobs; event streams like Kafka keep events for every interested service
- Kafka topics are append-only logs split into ordered partitions, with offsets as bookmarks
- Keys keep related events in order; consumer groups share partitions and scale consumers
- Publishing with KafkaTemplate and consuming with @KafkaListener, in the messaging package
- Designing clear events: good topic names, a consistent payload, no secrets, and the X-Correlation-Id header

9.2 Homework

18. Complete Exercises 4 and 5 and push your Library API, with its first Kafka events, to GitHub
19. Add a second event, library.book-returned, with its own producer method and listener
20. Draw a diagram of your system showing the API, Kafka, both topics, and every consumer group
21. Preview Week 17: Read about what happens when a Kafka consumer fails, and what a dead-letter topic is. Come with one question ready.

Coming Next — Week 17: Spring Kafka & Reliable Messaging

The Messaging module continues. Real systems must cope when things go wrong: a consumer crashes, a message cannot be processed, or the same event arrives twice. You will learn to send and receive JSON events, retry failed messages, route poison messages to a dead-letter topic, and build idempotent consumers — then complete the module practical: an event-driven notification service.

Closing Reflection

"Write the vision and make it plain on tablets, that he may run who reads it."

— Habakkuk 2:2 (NKJV)

God told Habakkuk to write the message down plainly, so that whoever read it — whenever they read it — could act on it. That is what a good event is: a clear, permanent record that others can read in their own time and run with. Write your events, your code, and your words plainly, so others can rely on them.

Glossary of Terms

Term	Definition
Asynchronous Processing	Doing work later, outside the request that triggered it
Broker	A Kafka server that stores partitions and delivers events
Consumer	A program that reads events from a topic and reacts to them
Consumer Group	Consumers sharing a group id, which split a topic's partitions between them
Correlation ID	An id carried from a request into every event and log line it causes, for tracing
Decoupling	Letting services work independently, without knowing about each other
Event	A record of something that has already happened, named in the past tense
Eventual Consistency	Different parts of a system catching up with each other a short time later
Kafka	A distributed event-streaming platform; the XENO platform's event backbone
KafkaTemplate	The Spring class used to send events to Kafka
Key	A value sent with an event that decides its partition and keeps related events in order
KRaft	The mode modern Kafka runs in, without a separate ZooKeeper server
Offset	The position number of an event within a partition
Partition	One ordered log within a topic; topics are split into partitions for scale
Producer	A program that publishes events to a topic
Retention	How long Kafka keeps events before deleting them
Topic	A named, append-only log of events, such as library.loan-created
@KafkaListener	The Spring annotation that marks a method as a consumer of a topic